Voice Assistance for Multimedia Control

Voice-Controlled Media Playback System (Python; tested on a laptop, Raspberry Pi-ready)

Software Architecture & Design Specification

Version 1.1, 29 October 2026 (Version 1.0: 28 September 2026)

Aaditya Vashisht: PES1UG24CS004
Aarohi Jaiswal: PES1UG24CS009
Abhay S: PES1UG24CS013

Revision History

Version	Date	Changes	Author
1.0	28 Sep 2026	Initial architecture and design, submitted with the SRS and Test Plan.	Aaditya Vashisht
1.1	29 Oct 2026	Updated to describe the system as built and tested in Sprints 1 and 2. Each platform change matches a deviation in the project README (D-x): • Platform: a laptop instead of a Raspberry Pi / ESP32 board (D-1); deployment view rewritten. • Media player: YouTube in a Chrome window controlled through Selenium (D-2). • ASR engine: the offline Vosk library with a closed grammar (D-3). • New components: Utterance Segmenter, Session Manager, Voice Assistant (orchestrator) and YouTube Player adapter. • New design sections: assistant status states (US-16, US-19, US-13), concurrency and timing, design decisions and known limitations (Previous, D-6). • All diagrams redrawn; the API table now uses the real function names in the code; traceability updated to SRS v1.1 and the Sprint 2 test results. The architectural pattern (layered pipe-and-filter), the four layers and the security design are unchanged. Changed text is marked “Revised in v1.1”.	Abhay S

3. Software Architecture & Design Specification

1. Introduction

1.1 Purpose

This document describes the software architecture and detailed design of the Voice Assistance for Multimedia Control system. It explains how the components work together, the architectural pattern used, and the detailed interaction and interface design that implements the requirements in the SRS.

1.2 Scope

This specification covers the internal software structure of the system running as a Python program on the user's computer (a laptop in this version; a Raspberry Pi is a supported target): how audio input flows through recognition, security validation and command execution, out to the media player and user feedback. The media player is YouTube in a Google Chrome window that the system opens and controls. The internal design of YouTube and Chrome is out of scope; only the interface used to control them is described. *[Revised in v1.1: D-1, D-2]*

1.3 References

- Software Requirements Specification (SRS), Voice Assistance for Multimedia Control, v1.1, 27 Oct 2026. *[Revised in v1.1: was v1.0]*
- Software Test Plan, Voice Assistance for Multimedia Control (Sprint 2), and the test log docs/test-log.md. *[Revised in v1.1: was v1.0]*
- IEEE Std 1016-2009, IEEE Standard for Information Technology – Software Design Descriptions.
- Alpha Cephei, Vosk Speech Recognition Toolkit, alphacephei.com/vosk (added in v1.1).
- Selenium Project, Selenium WebDriver Documentation, www.selenium.dev/documentation (added in v1.1).
- Project README (deviations D-1 to D-6) in the team GitHub repository (added in v1.1).

2. Architecture

2.1 Architectural Overview

The system is organized into four layers that mirror the flow of a voice command: a Hardware Layer (microphone, keyboard, speaker and terminal), a Recognition Layer (audio capture, utterance segmentation, wake-word detection, speech-to-text), a Control Layer (session management, command mapping, security validation, command execution and error handling) and an Output Layer (the media player interface and user feedback).

A single Voice Assistant object in the Control Layer coordinates the pipeline. It receives audio chunks, sends them to the Wake-Word Detector until a listening window is open, sends the next utterance to the ASR engine, and passes the resulting command to the Command Executor. It also owns the microphone privacy control (US-16), the status indicator (US-19) and the response-time log (NFR-1). *[Revised in v1.1: orchestrator added]*

2.2 Component Diagram and Description

Figure 1 shows the components grouped by layer. Arrows show the direction of data and control flow; the labels are the function calls or data passed. Each box names the source file that implements it.

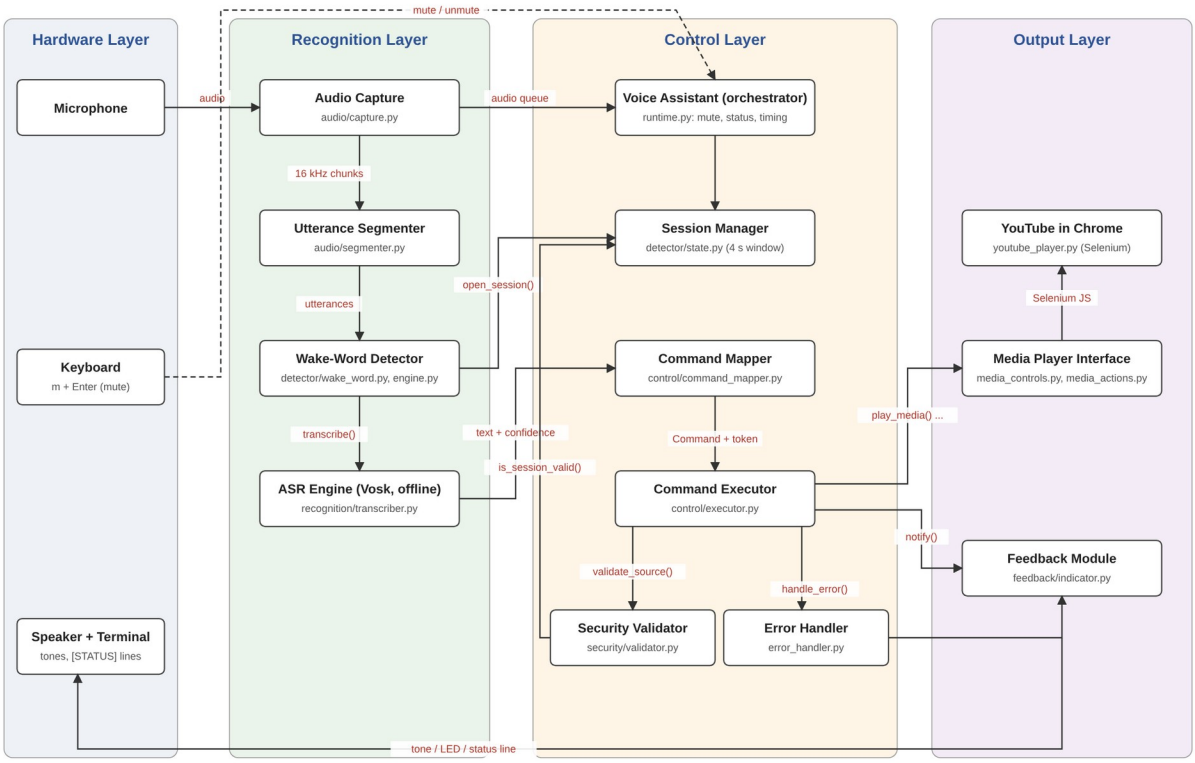


Figure 1: Component Diagram — Voice-Controlled Multimedia System (revised in v1.1)

Component	Responsibility
Audio Capture Component	Opens the microphone (16 kHz, 16-bit mono) and delivers ~64 ms chunks to an in-memory queue. Detects a missing or failed microphone and suspends capture until it returns (US-13). Can be stopped and restarted for mute (US-16). *[Revised in v1.1: laptop microphone]*
Utterance Segmenter	Splits the audio stream into utterances using an energy threshold: an utterance ends after 0.5 s of silence (maximum 3 s). Audio is discarded once it has been handed on. *[Revised in v1.1: new]*
Wake-Word Detector	Transcribes each utterance and opens a listening window only if the exact phrase “hey media” appears as whole words with enough confidence (SR-1). Words said after the wake word in the same breath (“hey media play”) are passed on as the command. *[Revised in v1.1: transcript-based verification]*
ASR / Speech-to-Text Engine	Vosk offline recognizer with a closed grammar: the wake phrase, the command phrases and [unk] for anything else. Runs on the device; no audio leaves the computer (SR-2). A Google cloud recognizer exists only as an explicitly selected option. *[Revised in v1.1: D-3]*

Component	Responsibility
Session Manager	One shared state machine (IDLE / LISTENING). Opens a 4-second listening window with a random session token, closes it after one command or on timeout, and tells the Feedback Module about state changes. *[Revised in v1.1: new]*
Voice Assistant (orchestrator)	Runs the main loop, routes audio to the detector or the command path, enforces one command per wake word, applies mute/unmute typed as m + Enter, computes and prints the status line, and logs response time from end of speech. *[Revised in v1.1: new]*
Command Mapper	Matches the recognized text against the command vocabulary (Play, Pause, Resume, Next, Previous, Volume Up, Volume Down, plus a few synonyms such as “skip”) and produces a Command object carrying the session token, or None.
Security Validator	Final gate before execution: the command’s session token must match the open, unexpired session, and its confidence must be at least 0.5 (SO-1, SR-1). Re-checked after recognition, so a command finished after the window closed is rejected. *[Revised in v1.1: confidence check added]*
Command Executor	Calls the Security Validator, then the matching media function; sends ok/error feedback; passes any exception to the Error Handler so a media failure never crashes the loop.
Error Handler	Logs every error with its type, keeps a short history and triggers the matching error tone; never raises further.
Media Player Interface	media_controls.py exposes play_media(), pause_media(), resume_media(), next_track(), previous_track() and adjust_volume(direction). Each returns True only after checking that the player really changed. media_actions.py binds them to command types and reports any that are missing at start-up. *[Revised in v1.1: real function names]*
YouTube Player adapter	Opens one dedicated Chrome window through Selenium at start-up and runs small JavaScript snippets against the page’s video element and player buttons. *[Revised in v1.1: new, D-2]*
Feedback Module	Gives audible tones and LED-style colour states (blue: listening, green: done, red: rejected, yellow: muted) and the console messages that confirm each result (FR-10).

2.3 Architectural Pattern

The system follows a Layered (Pipe-and-Filter) architecture. Audio flows in one direction through a pipeline of filters (capture, segmentation, wake-word detection, recognition, mapping, validation, execution), each stage consuming the previous stage’s output. This pattern fits a problem that is a linear transformation of a signal (raw audio) into a discrete action (a media command), and keeps each stage independently testable.

Two supporting patterns are used in v1.1: a central orchestrator (the Voice Assistant) that owns the main loop, and a small state machine (the Session Manager) that the detector and the validator share, so both always see the same listening window. The media functions are plugged in through an adapter, so the pipeline is independent of the player (NFR-6). *[Revised in v1.1: supporting patterns named]*

2.4 Traceability to Requirements

Each component is traced to the requirement(s) it satisfies (SRS v1.1) and to the evidence that verifies it.

Component(s)	Requirement	Verified by (Sprint 2)
Audio Capture, Utterance Segmenter, Wake-Word Detector	FR-1, SR-1	Unit tests; live checks in the test log
Command Mapper, Command Executor, Media Player Interface	FR-2 to FR-8	Unit tests; live checks (FR-6 limited, see Section 4)
Command Mapper, Error Handler	FR-9	Unit tests; live rejection checks (US-14)
Feedback Module	FR-10	Unit tests; live checks (US-09)
Whole pipeline, response-time log	NFR-1	US-11: 712 ms average, slowest 1,160 ms over 40 commands
Vosk ASR with closed grammar	NFR-2	US-11: 46 of 51 attempts correct (90.2%)
Bounded histories, microphone retry, main loop	NFR-3	US-10: continuous run of more than 2 hours (evidence in review)
Command vocabulary and synonyms	NFR-4	Usability test with two first-time users
Python-only design, adapter for the player	NFR-5, NFR-6	Runs on Windows and macOS laptops; CI runs on Linux
Session Manager, Security Validator	SO-1, SR-1	Unit tests; live false-trigger checks
Local Vosk ASR, in-memory audio only	SO-2, SR-2	US-17: network check (no connections opened by the assistant); automated test with the network blocked
Voice Assistant mute control and status line	US-16, US-19	Unit tests; live checks in the test log

2.5 Security Architecture

Security is addressed at several points in the pipeline rather than as an afterthought:

- Wake-word gating: a listening window opens only when the recognized text contains the exact phrase “hey media” as whole words with confidence of at least 0.65. Similar words or command words on their own do nothing (SO-1, SR-1). *[Revised in v1.1: verified by transcript]*
- Session token: every command carries the token of the window it was heard in. The window lasts 4 seconds and accepts one command; the token is then discarded, so a command cannot be replayed into a later window. *[Revised in v1.1: new]*
- Final gate: the Security Validator runs immediately before the Command Executor calls the media player, re-checks the session after recognition and rejects low-confidence results, so no command reaches the player without passing validation.
- Local-only processing: the Vosk recognizer and all downstream components run on the computer; audio is held only in memory and is never written to disk or sent over the network (SO-2, SR-2). The cloud recognizer must be selected explicitly with --backend google. *[Revised in v1.1: D-3]*

- Privacy control: typing m + Enter closes the microphone stream itself, cancels any open window and discards audio captured before the mute (US-16). *[Revised in v1.1: new]*

2.6 Deployment View

All components run in a single Python process on one computer (Windows, macOS or Linux; Python 3.9 or newer). Inside that process the microphone callback thread puts audio into a bounded queue, a keyboard thread reads typed controls, and the main loop does all processing, so the pipeline itself is single-threaded. The process starts one Chrome window through ChromeDriver. Speech recognition needs no network; the network is used only by Chrome to stream YouTube videos. The same program can run on a Raspberry Pi (Linux) without code changes; ESP32 firmware is out of scope. *[Revised in v1.1: D-1, D-2]*

3. Design

This section presents the interaction-level design: two UML sequence diagrams (the main flow and the error flow), the assistant status states, the internal API, error handling, data design, and concurrency and timing.

3.1 Sequence Diagram 1 – Play Command Execution Flow

Figure 2 shows the end-to-end flow when a user says “Hey Media”, then “Play” within the 4-second window, including the security check before execution and the feedback returned to the user. The same flow applies, with the command substituted, for Pause, Resume, Next, Previous, Volume Up and Volume Down (FR-3 to FR-8). When the user says the wake word and command in one breath (“Hey Media play”), the command is taken from the same utterance and the flow continues at map_command().

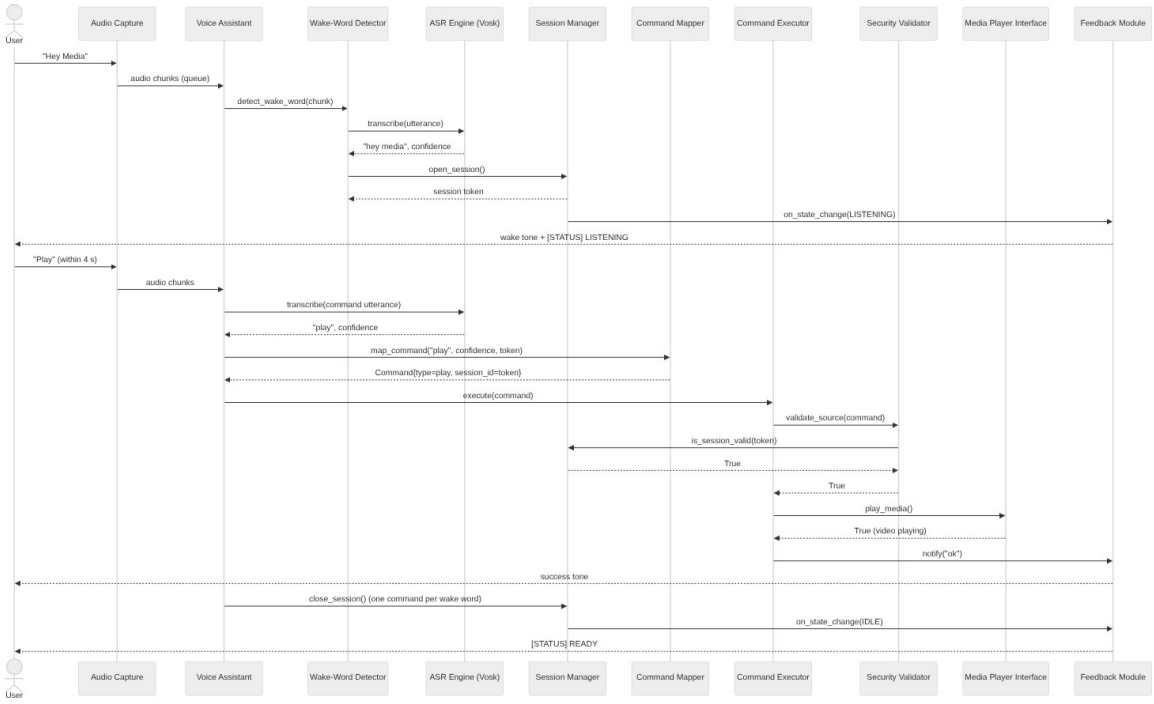


Figure 2: Sequence Diagram 1 — Play Command Execution Flow (revised in v1.1)

3.2 Sequence Diagram 2 – Unrecognized Command / Error Handling Flow

Figure 3 shows the flow when the utterance after the wake word is not a command in the vocabulary. The Command Mapper returns None, the Error Handler logs the event and plays the error tone, the window closes, and the media player is never called (FR-9).

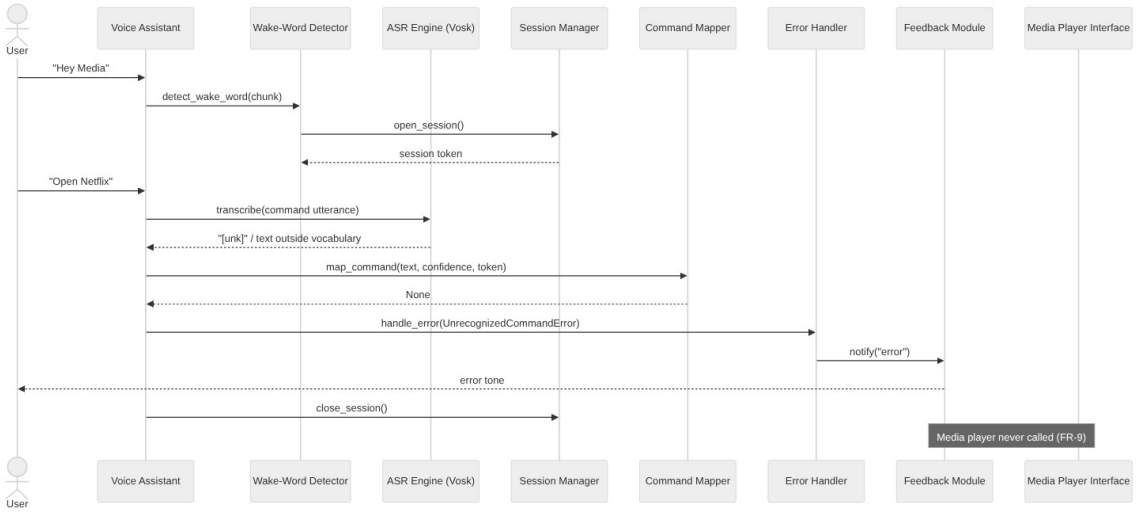


Figure 3: Sequence Diagram 2 — Unrecognized Command / Error Handling Flow (revised in v1.1)

3.3 Assistant Status States (added in v1.1)

Figure 4 shows the status the user sees on the terminal (US-19). The status is computed from the real state each time, and a status line is printed only when it changes. Muted always wins over Unavailable, and both cancel an open listening window.

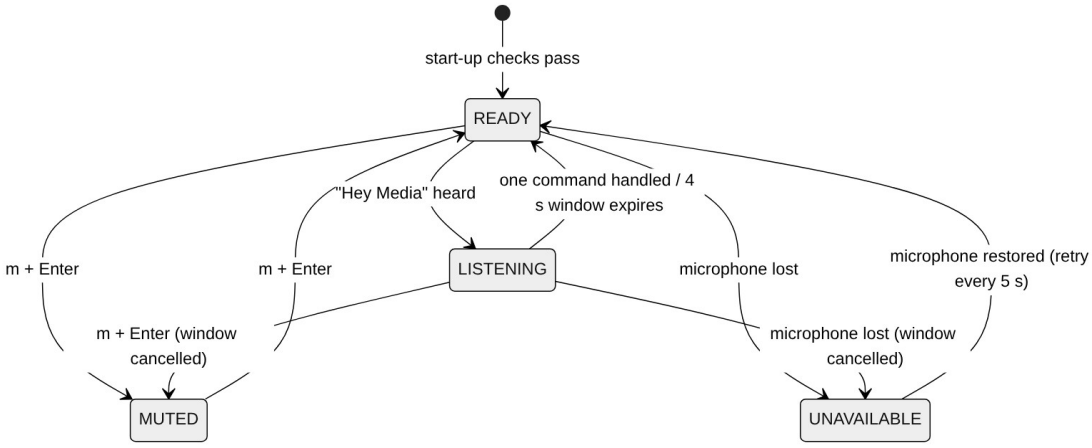


Figure 4: State Diagram — Assistant Status (READY, LISTENING, MUTED, UNAVAILABLE)

3.4 API Design

The table below defines the internal API between components. All calls are local function calls in the same process; no network API is needed for core functionality. *[Revised in v1.1: real names from the code]*

Function	Owner	Parameters	Returns	Notes
detect_wake_word(audio_chunk)	Wake-Word Detector	audio_chunk: bytes	bool	Called for every chunk while no window is open; opens the session when true.
transcribe(pcm)	ASR Engine	pcm: bytes	Transcript {text, confidence} or None	Called once per complete utterance.
open_session() / close_session()	Session Manager	none	token: str / none	4-second window; one command per wake word.
is_session_valid(token)	Session Manager	token: str	bool	False if no window is open, it has expired, or the token differs.
map_command(text, confidence, session_id)	Command Mapper	text: str, confidence: float, session_id: str	Command or None	None when the text is not exactly a known command.
validate_source(command)	Security Validator	command: Command	bool	Checks the session token and minimum confidence.
execute(command)	Command Executor	command: Command	ExecutionResult {success, message}	Validates, calls the media function, gives feedback.
play_media(), pause_media(), resume_media(), next_track(), previous_track(), adjust_volume(direction)	Media Player Interface	direction: 'up' | 'down' (volume only)	bool	True only if the player state really changed; volume steps by 10%.
handle_error(error)	Error Handler	error: Exception	none	Logs and plays the matching error tone; never raises.
notify(status)	Feedback Module	status: 'ok' | 'error' | 'mic_error' | 'muted' | 'unmuted'	none	Tone, colour state and message for the user.
set_muted(muted), toggle_mute()	Voice Assistant	muted: bool	none / bool	Typed as m + Enter; stops and restarts the microphone (US-16).

Function	Owner	Parameters	Returns	Notes
status, status_message()	Voice Assistant	none	AssistantStatus / str	READY, LISTENING, MUTED or UNAVAILABLE (US-19).
timing_summary()	Voice Assistant	none	{count, average_ms, max_ms}	Printed on exit; used for NFR-1 evidence (US-11).

3.5 Error Handling

All errors go through the central Error Handler so that failure behaviour is consistent and never causes an unintended media action. The handled conditions are:

Error condition	Detected by	System response	Related req.
No wake word	Wake-Word Detector	The system stays READY; nothing downstream runs.	SR-1
Command outside the vocabulary	Command Mapper returns None	Error is logged, error tone, window closes; no media action.	FR-9
Speech not understood	ASR returns no result	“Didn’t catch that, please repeat”, error tone, back to READY.	FR-9, NFR-2
Window expired before the command finished *[Revised in v1.1: new]*	Session Manager	Error is logged and the window closes; a late command is rejected by the validator.	SR-1
Security validation fails	Security Validator	The media function is never called; a security event is logged.	SO-1, SR-1
Media action fails or Chrome is closed	Media function returns False or raises	Error tone; the assistant keeps running and later commands still work.	FR-10, NFR-3
Media function missing *[Revised in v1.1: new]*	media_actions at start-up	Warning at start-up; that command is rejected with an error tone.	NFR-6
Microphone missing or disconnected	Audio Capture Component	Status shows MICROPHONE UNAVAILABLE; capture is retried every 5 seconds and resumes automatically. *[Revised in v1.1: retry]*	NFR-3, US-13
Processing falls behind *[Revised in v1.1: new]*	Audio queue full	The oldest audio chunk is dropped so the system stays responsive.	NFR-1

3.6 Data Design

- Audio: a rolling in-memory buffer of at most 3 seconds and a bounded queue of 200 chunks. Utterance audio is discarded as soon as it has been transcribed; nothing is written to disk (SR-2). *[Revised in v1.1: sizes stated]*

- Command object: {type: str, confidence: float, timestamp: datetime, session_id: str | None}. session_id defaults to None, so a command built without a token is always rejected. No user-identifying data is stored. *[Revised in v1.1: session_id added]*
- ExecutionResult: {success: bool, message: str, command_type: str, executed_at: datetime}. *[Revised in v1.1: new]*
- Histories of results, response times and statuses keep at most 200 entries each, so memory does not grow during long runs (NFR-3). *[Revised in v1.1: new]*

3.7 Concurrency and Timing (added in v1.1)

- The microphone callback only puts chunks into the queue; the keyboard thread only puts typed text into a control queue. All decisions are made on the main loop, so components need no locks.
- Response time (NFR-1) is measured from the end of the user's speech (the end of the utterance minus the trailing silence) to the moment the media function returns, and is logged as a [TIMING] line for every command.
- When the queue is empty for 0.2 s the loop still checks the session, so an expired window is noticed without new audio.

4. Design Decisions, Deviations and Known Limitations (added in v1.1)

The design choices below differ from version 1.0 or from the SRS v1.0 platform. Each is recorded in the project README and will be explained at the final demo.

ID	Decision	Reason and effect on the design
D-1	Laptop instead of Raspberry Pi / ESP32	No board was available to the team. The software is pure Python and runs unchanged on a Pi (Linux); only the deployment view changed.
D-2	YouTube in Chrome via Selenium	A media player every team member could run. Added the YouTube Player adapter; media functions verify the result in the page.
D-3	Vosk offline recognizer	Meets SR-2 (local processing) with a small model; the closed grammar improves accuracy for the 7 commands.
D-4	US-09 and US-13 delivered early	Feedback and microphone error handling were built with the Sprint 1 pipeline; no design impact.
D-5	NFR-3 reduced to 2 hours	Approved by the instructor; bounded histories and microphone retry support longer runs.
D-6	FR-6 Previous limitation	YouTube goes to the previous video only within about 4 seconds of a video starting; after that it restarts the current video, and previous_track() reports failure. Not fixed by team decision; documented for the final demo.